Rules as Code

A Survey with Emphasis on the Opportunities Created by AI

Research survey

18 July 2026

Abstract

Rules as Code (RaC) makes laws, regulations, policies, and standards available in governed, machine-consumable forms. This survey reviews the field's concepts, methods, international activity, technical ecosystem, limitations, and maturity. It pays particular attention to artificial intelligence: AI can lower the cost of extracting, formalizing, testing, maintaining, simulating, and explaining rules, while deterministic rule models can in turn constrain AI agents. The strongest design is hybrid: probabilistic AI around a traceable, testable, deterministic rule core, with accountable human interpretation and review.

Executive summary

Rules as Code is the practice of creating a machine-consumable representation of laws, regulations, policies, or standards—ideally alongside the authoritative natural-language text and through the same policy-development process.

The essential idea is not that software replaces law. Governments and regulated organizations already translate rules into software whenever they build tax, benefits, licensing, compliance, permitting, or reporting systems. RaC makes that translation explicit, reusable, testable, traceable, and subject to governance.

The OECD's influential formulation proposes an official machine-consumable version of suitable government rules alongside their human-readable counterpart. This could reduce repeated, inconsistent interpretations by agencies and regulated entities while accelerating public-service delivery. The OECD also emphasizes unresolved questions concerning legal status, accountability, and applicability to rules involving discretion [1].

AI changes RaC primarily by reducing the cost of constructing and maintaining formal rule systems. It does not eliminate the need for those systems. Language models can help extract candidate rules, generate formal representations and tests, detect inconsistencies, analyze amendments, and explain deterministic results. Conversely, executable rules can define permissions, prohibitions, approvals, and audit requirements for AI agents.

The central recommendation of this survey is a hybrid architecture:

1. authoritative legal sources;
2. AI-assisted extraction, comparison, and drafting;
3. a human-governed semantic and executable rule model;
4. static analysis, simulation, and approved test suites;
5. a deterministic engine and versioned API; and
6. public services, compliance systems, and AI assistants consuming the governed result.

An unconstrained language model should not be the authoritative decision engine for legally consequential decisions. Its outputs are probabilistic, difficult to reproduce, vulnerable to prompt manipulation, and capable of producing plausible but unsupported explanations.

1. Defining Rules as Code

RaC sits on a spectrum. Only the later stages are normally considered Rules as Code; a searchable PDF is digitized law but is not machine-consumable as a rule system.

Level	Representation	Machine capability
Digitized law	PDF or HTML	Display and search text
Structured law	XML, Akoma Ntoso, metadata	Identify provisions, amendments, and citations
Semantic law	Ontologies and knowledge graphs	Connect concepts, entities, and dependencies
Decision models	Tables, trees, structured rule statements	Analyze eligibility and obligations
Executable rules	DSLs, logic programs, conventional code	Calculate or decide concrete cases
Rules as infrastructure	Versioned packages and APIs	Embed official calculations in many services

An open and accountable RaC model adds several requirements:

- every coded rule can be traced to authoritative source provisions;
- interpretive choices and assumptions are visible;
- lawyers, policy owners, affected communities, and technologists can review the representation;
- rules, tests, and provenance are versioned together;
- the formal representation is separable from any one service or user interface; and
- human-readable law remains available and, unless legislation provides otherwise, authoritative.

The European Commission's Interoperable Europe work identifies the usually hidden "middle layer"—analysis, interpretation, formalization, validation, and governance—as RaC's central leverage point [2]. RaC is therefore as much a governance and institutional-design practice as it is a technical practice.

Related but distinct concepts

Machine-readable law commonly means that a document's structure can be parsed. **Machine-consumable law** means that a system can directly use its meaning or logic. **Computational law** is the broader use of computation in legal analysis and delivery. **Policy as Code** often refers to executable organizational or infrastructure controls, such as authorization and configuration rules. **Code is law** describes software's ability to regulate behavior; **law as code** or **rules as code** describes the formalization of legal or policy rules. These fields overlap, but they should not be conflated.

2. Why the field matters

2.1 Consistency and reuse

Multiple agencies, vendors, banks, payroll providers, and advisers currently translate the same rules independently. Implementations can diverge. A governed rule model can become a shared reference implementation and reduce duplicated interpretation.

2.2 Faster, more reliable service delivery

A change to a benefit threshold or tax formula could flow into calculators, case-management systems, websites, and authorized third-party products from one versioned source. This can shorten the gap between a policy change and its correct operational implementation.

2.3 Testability before enactment

Formalization exposes missing definitions, circular dependencies, boundary cases, conflicting provisions, and unstated assumptions. Proposed policies can be simulated against representative or synthetic populations before legislation is finalized.

The Catala research team applied its law-oriented language to French family-benefit rules and United States tax legislation and reported finding a defect in an official implementation during formalization [3]. The important benefit was not merely executable code: the formalization created a shared medium for legal and technical specialists to expose misunderstandings.

2.4 Transparency and explainability

A deterministic engine can return the result, inputs, rules fired, intermediate calculations, source provisions, model version, and effective date. That is generally easier to audit than either opaque legacy application code or a probabilistic AI answer.

2.5 Access to entitlements and obligations

RaC can power calculators, personalized guidance, proactive benefits, compliance checks, and "what-if" tools. Open publication lets civil society and regulated parties test whether operational implementations reflect the published law and stated policy intent.

2.6 Compliance by design

Enterprises can consume governed rule models instead of manually encoding every regulatory change. Attractive domains include tax, payroll, financial services, environmental permitting, construction, procurement, and cross-border trade.

3. Approaches and technologies

There is no single dominant RaC language or architecture. The most important design choice is often the governance and intermediate representation, rather than the final execution language.

3.1 Decision models and rule statements

New Zealand's Better Rules methodology brings policy, legal, service-design, and technical specialists together to produce concept models, decision models, structured rule statements, and testable implementations. Its distinctive ambition is to develop natural-language legislation and software representations concurrently, preserving intent earlier in the lifecycle [4, 5].

3.2 Domain-specific languages and logic systems

Languages designed for legal computation can preserve relationships between prose and executable logic.

- **Catala** is a literate programming language for tax and social-benefit computations. It explicitly models defaults and exceptions common in legislation, and its compiler has a formally verified core [3].
- **Blawx and DataLex-style systems** use logic programming to represent propositions, exceptions, and conclusions and emphasize explainable results.
- Other approaches employ LegalRuleML, defeasible logic, Datalog, Prolog, and jurisdiction-specific domain languages.

3.3 General rule engines and decision standards

Organizations also use DMN decision tables, business-rule systems, production-rule engines, declarative policy languages, or conventional programming languages with strong traceability conventions. These can be practical, but executable software is not automatically good RaC. Without source mapping, legal review, versioning, and transparent interpretation, it may simply be undocumented application logic.

3.4 Semantic representations

Knowledge graphs, controlled vocabularies, ontologies, European Legislation Identifier metadata, and structured legislative XML help machines resolve definitions, cross-references, amendments, authority, jurisdiction, effective periods, and dependencies. They complement executable logic, especially where a large interconnected body of law cannot be reduced to one stand-alone rules file.

3.5 APIs and reusable models

OpenFisca is a prominent open-source ecosystem for modelling taxes and social benefits and exposing computations through APIs [6]. Models can support eligibility tools, distributional analysis, research, and

operational services. An OpenFisca model is not inherently an authoritative expression of law; authority depends on its publisher, mandate, and governance.

4. International landscape

4.1 New Zealand

New Zealand's Better Rules work is foundational. Its discovery research found that policy intent can degrade through a chain from legislation to interpretation, business rules, requirements, and application code. Multidisciplinary co-drafting and shared models are intended to shorten that chain [4, 5].

4.2 France and OpenFisca

OpenFisca originated in France and has been adopted for tax and benefit modelling in multiple jurisdictions. It demonstrates reusable public computation infrastructure: one governed model can support calculators, simulations, research, and services.

4.3 European Union and member states

Europe is moving from isolated pilots toward an ecosystem around digital-ready legislation, interoperability, and citizen-centric services. The Commission-supported GovTech4All pilot combines guidelines, AI tools, RaC APIs, a secure framework, and proofs of concept with public institutions [7].

A Commission-supported Rules as Code Europe event in March 2025 brought together more than 100 specialists from public administrations, academia, and industry [8]. Denmark's digitally ready legislation program evaluates whether proposed laws can be implemented digitally, while the Netherlands has long used rule-based systems in high-volume administration. These initiatives overlap with RaC even where terminology differs.

4.4 Canada

The Canadian Digital Service explored RaC for navigating mining regulation. Its account illustrates both the opportunity—shared and current regulatory logic—and the difficulty created by a complex, frequently changing regulatory landscape [9].

4.5 Australia

Australian researchers and governments have been active in logic-based legal representation, tax and benefits, cost-benefit analysis, explainability, and the rule-of-law implications of RaC. Australian scholarship highlights that formalization is a kind of legal translation involving choices, rather than a neutral transcription of obvious meaning [10].

The OECD concludes that adoption has grown but that current techniques work best on relatively straightforward provisions rather than all law indiscriminately [11].

5. Selecting suitable rules

RaC works best where a rule has well-defined inputs, explicit thresholds or formulas, bounded exceptions, objective dates and classifications, a repeatable decision procedure, and a result testable against examples.

Strong candidates include taxation, payroll, benefits, fees, grants, pensions, procurement thresholds, filing deadlines, licensing prerequisites, quantitative reporting obligations, and some building or environmental checks.

RaC is less suitable as a complete substitute for judgment where decisions depend heavily on terms such as "reasonable," "proportionate," or "in the public interest"; credibility and evidentiary weight; balancing rights; equitable exceptions; or evolving judicial interpretation.

Open-textured provisions can still be represented partially. Code can support navigation, issue spotting, process control, or recording the exercise of discretion. It must not falsely imply that a contested judgment has one computationally inevitable answer.

6. Opportunities created by AI

AI's most credible role is to reduce the cost of producing and governing formal systems while leaving authoritative execution to verified artifacts.

6.1 Candidate rule extraction

Language models can identify definitions, obligations, prohibitions, actors, affected classes, conditions, exceptions, formulas, thresholds, dates, cross-references, delegated powers, and discretionary language. Instead of beginning with a blank page, specialists can review a proposed structure.

The output should be a candidate model with provision-level citations, confidence or uncertainty markers, and explicit interpretive questions—not automatically published law.

6.2 Prose-to-model translation

Models can propose decision tables, controlled natural language, knowledge-graph triples, schemas, API contracts, or executable OpenFisca, Catala, DMN, and logic code.

Experiments in United States public-benefits policy have tested chat-based policy interpretation, machine-readable summaries, fine-tuning, retrieval-augmented generation, and policy-to-code generation [12]. They show the potential to accelerate eligibility-system implementation, but not that unattended conversion is safe.

6.3 Test and counterexample generation

AI may be more immediately valuable for verification than initial translation. It can propose statutory examples, boundary values around thresholds, combinations of exceptions, temporal cases around commencement or expiry, property-based tests, adversarial scenarios, and counterexamples where implementations disagree.

Generated cases can be executed against a deterministic system. The expected legal outcome must still derive from an approved oracle: an authoritative example, reviewed interpretation, legacy result confirmed as correct, or independently validated calculation.

6.4 Drafting-time quality analysis

Comparing prose, models, code, and test behavior can flag undefined terms, inconsistent terminology, unreachable clauses, overlapping categories, exceptions that swallow a rule, missing precedence, circular references, and unexpected distributional effects. This moves feedback earlier, when policy is less costly to change.

6.5 Amendment and dependency analysis

When an amendment is proposed, AI plus a legal knowledge graph can suggest affected provisions, rules, datasets, forms, guidance, APIs, tests, services, and downstream organizations. Dependency analysis and

accountable owners must verify the proposed impact set.

6.6 Policy simulation

A deterministic RaC model supplies reliable calculations while statistical and generative systems help explore alternative thresholds, phase-outs, definitions, or exceptions. Analysis can cover fiscal cost, benefit take-up, exclusion, distribution among demographic groups, administrative burden, and sensitivity to assumptions.

AI should support political judgment rather than silently optimize policy against a single metric.

6.7 Accessible, grounded explanations

An LLM can turn an execution trace into plain language, multiple languages, or accessible formats. For example, it can explain that a condition was not met, identify the recorded input, and link the governing provisions. The conclusion, trace, and citations should be supplied by the deterministic engine; the model explains evidence rather than inventing a legal outcome.

6.8 Compliance copilots

An enterprise assistant can combine retrieval over authoritative sources, executable regulatory rules, organizational controls, case data, and an audit trail. The assistant addresses "What may apply?" while the rule engine answers "Does this case satisfy the approved formal conditions?" This hybrid is more defensible than retrieval-augmented generation alone.

6.9 Executable constraints for AI agents

RaC can govern AI as well as be created with AI. Formal rules can prohibit payments above a threshold without approval, restrict sensitive records, require human judgment for discretionary factors, log the rule version authorizing every action, and halt execution when evidence is missing.

This reciprocal relationship is strategically important:

- AI helps people produce and maintain Rules as Code.
- Rules as Code helps people constrain and audit AI.

6.10 Continuous regulatory monitoring

AI can monitor newly published legislation, decisions, guidance, and standards and propose updates to affected artifacts. A governed release process should still require legal-owner approval, regression tests, effective-date checks, signed releases, and reproducible builds.

7. Why an LLM should not be the rule engine

Generative models are probabilistic and sensitive to prompts and context. Law also contains ambiguity, but that does not make uncontrolled probabilistic execution desirable. An LLM used as the final decision-maker creates several problems:

- materially identical cases may receive different answers;
- provisions and citations may be invented or misapplied;
- prompt injection or irrelevant context may change behavior;
- model updates can silently alter outcomes;
- comprehensive path testing is difficult;
- explanations may be plausible without describing the actual computation;

- reproducing a historical decision becomes difficult; and
- legal authority and accountability are unclear.

The strongest pattern is **neuro-symbolic**: AI handles language, discovery, candidate generation, and explanation; symbolic models handle approved calculations and constraints; people retain responsibility for interpretation, exceptions, and publication; and provenance plus tests connect each layer.

Interoperable Europe explicitly characterizes RaC as knowledge-driven or symbolic AI, contrasting it with data-driven machine learning and generative AI [13].

8. Risks and safeguards

False authority

Users may assume an official-looking answer is legally binding. Outputs must identify whether a model is authoritative, advisory, or experimental; its jurisdiction and date; its sources; and available review or appeal routes.

Interpretive capture

Encoding one interpretation can hide legitimate alternatives and turn a contestable policy choice into apparent technical fact. Alternative interpretations, assumptions, and unresolved questions should be first-class artifacts.

Automation bias and procedural fairness

Officials and citizens may defer to a result even when the input is wrong or discretion is required. Systems need visible exception, correction, human-review, reason-giving, and appeal paths.

Equality, accessibility, and participation

Formal consistency can reproduce inequity in source rules or data. Simulation and testing should cover protected groups, atypical households, missing documents, disability, language, and digital exclusion. Affected communities require meaningful opportunities to inspect and challenge assumptions.

Temporal complexity

Decisions may depend on the law at the event, filing, assessment, or appeal date. Repositories need explicit temporal semantics, immutable releases, amendment relationships, and reproducible historical execution.

Institutional authority and separation of powers

If an executive agency publishes executable interpretations, those artifacts must not displace the legislature's text or courts' interpretive role without explicit constitutional authority. Formal models should disclose who approved each interpretation and under what mandate.

Security and strategic behavior

A centralized rules API may become critical infrastructure. Risks include unauthorized rule changes, supply-chain compromise, denial of service, data leakage, and gaming of decision boundaries. Security controls must coexist with transparency; publication of logic does not require publication of personal data or exploit-sensitive enforcement tactics.

Accountability for AI assistance

AI-generated proposals should retain the model and configuration, structured task or prompt, retrieved sources, generated diff, reviewer identities, approvals, tests, and deployment signature.

For legally consequential AI, contemporary governance expectations include traceability, documentation, human oversight, robustness, cybersecurity, and accuracy. The European Union's AI Act provides a prominent risk-based example [14].

9. Maturity assessment

Capability	Maturity	Assessment
Hand-authored tax and benefit models	High	Demonstrated operationally
Decision tables and deterministic APIs	High	Established technology
Multidisciplinary RaC drafting	Medium	Strong pilots; limited institutionalization
Structured legislation and legal graphs	Medium	Standards exist; coverage varies
AI-assisted clause extraction	Medium	Useful under review
AI-generated draft executable rules	Medium-low	Promising but unsafe unattended
AI-generated tests and edge cases	Medium-high	Strong near-term use with a verified oracle
Automated amendment-impact analysis	Medium	Depends on provenance and graph quality
Fully automated law-to-production pipeline	Low	Technically and institutionally unsafe
LLM as authoritative decision engine	Inappropriate in most consequential uses	Poor assurance and reproducibility
RaC constraints for autonomous agents	Emerging	Significant opportunity; fragmented standards

10. Adoption roadmap

1. **Select a bounded case.** Choose deterministic, high-volume rules with measurable implementation pain, stable data definitions, and an accountable owner.
2. **Establish governance.** Decide who owns interpretation, approves releases, resolves disputes, and handles review or appeals.
3. **Create a canonical intermediate model.** Separate legal meaning from one programming language, application, or vendor.
4. **Make provenance structural.** Link every condition and calculation to provisions, definitions, interpretation notes, authority, and effective dates.
5. **Develop a gold test corpus.** Include authoritative examples, confirmed historical cases, boundary values, exceptions, temporal transitions, and fairness-relevant scenarios.
6. **Use AI as a copilot.** Let it extract, draft, compare, and propose tests. Require grounded output and human approval for substantive interpretation.
7. **Compile or map to deterministic execution.** Require versioning, reproducibility, complete traces, and signed releases.

8. **Publish appropriately.** Release models, source mappings, tests, APIs, version history, legal status, and limitations where security and privacy allow.
9. **Run in shadow mode.** Compare the new model with current operations before it affects rights or obligations.
10. **Measure outcomes.** Track interpretation defects, implementation time, disagreement, appeals, exclusion errors, update latency, reuse, and maintenance cost.

Conclusion

Rules as Code is best understood as governance reform supported by technology, not primarily as a coding technique. Its value comes from making the translation between policy and implementation visible and testable.

AI materially expands the opportunity by making formalization, testing, change analysis, simulation, and explanation cheaper. It simultaneously makes provenance and deterministic verification more important. The promising future is not "ask an LLM what the law says." It is a governed system in which AI helps people construct and maintain transparent formal rules, approved deterministic models produce reproducible outcomes, and every consequential result preserves a path back to authoritative law and accountable human review.

References

1. OECD, *Cracking the Code: Rulemaking for Humans and Machines* (2020). https://www.oecd.org/content/dam/oecd/en/publications/reports/2020/10/dechiffrer-le-code_d56cab77/3afe6ba5-en.pdf
2. Interoperable Europe, "Rules as Code—An Open Approach" (2025). https://interoperable-europe.ec.europa.eu/collection/eugovtech/document/rules-code-open-approach
3. Denis Merigoux, Nicolas Chataing, and Jonathan Protzenko, "Catala: A Programming Language for the Law" (2021). https://arxiv.org/abs/2103.03198
4. New Zealand Digital Government, *Better Rules for Government Discovery Report* (2018). https://www.digital.govt.nz/dmsdocument/95-better-rulesfor-government-discovery-report/html
5. Better Rules—Better Outcomes, "What Better Rules—Better Outcomes Is All About." https://www.betterrules.govt.nz/about
6. OpenFisca, "Write Rules as Code." https://openfisca.org/en/
7. Interoperable Europe, "Citizen Centric Rules as Code." https://interoperable-europe.ec.europa.eu/collection/eugovtech/citizen-centric-rules-code
8. Interoperable Europe, "Rules as Code Europe" (2025). https://interoperable-europe.ec.europa.eu/collection/eugovtech/news/rules-code-europe
9. Canadian Digital Service, "Mining for Ideas to Simplify a Complex Process" (2023). https://digital.canada.ca/2023/10/12/mining-for-ideas-to-simplify-a-complex-process/
10. Olivia Hamlyn, "Fidelity in Legal Coding: Applying Legal Translation Frameworks to Address Interpretive Challenges" (2024). https://www.tandfonline.com/doi/full/10.1080/13600834.2024.2312620
11. OECD, *Global Trends in Government Innovation 2023*, "Rules as Code." https://www.oecd.org/en/publications/global-trends-in-government-innovation-2023_0655b570-en/full-report/component-3.html
12. Digital Benefits Network and Massive Data Institute, "AI-Powered Rules as Code: Experiments with Public Benefits Policy" (2024). https://digitalgovernmenthub.org/publications/ai-powered-rules-as-code-experiments-with-public-benefits-policy/
13. Interoperable Europe, "The Thing with Rules" (2024). https://interoperable-europe.ec.europa.eu/collection/eugovtech/news/rules-code-rac
14. European Commission, "AI Act." https://digital-strategy.ec.europa.eu/en/policies/regulatory-framework-ai
15. Lisa Burton Crawford, "Rules as Code and the Rule of Law," *Public Law* 2023(3), 402. https://papers.ssrn.com/sol3/papers.cfm?abstract_id=5184244
16. Andrew Mowbray, Philip Chung, and Graham Greenleaf, "Representing Legislative Rules as Code: Reducing the Problems of Scaling Up" (2021). https://papers.ssrn.com/sol3/papers.cfm?abstract_id=3981161

Research note

This survey was prepared on 18 July 2026 using the linked government, intergovernmental, project, and academic sources. It is a field survey rather than legal advice. Programs, implementations, and regulatory requirements continue to evolve; current details should be verified before making a specific legal or operational decision.